Aula prática: consumindo APIs públicas com Python (PokeAPI e REST Countries)

Na Unidade 2, usamos a biblioteca `requests` para consumir a API de dispositivos que roda na nossa própria máquina (`http://127.0.0.1:5000`). Nesta aula, vamos aplicar exatamente as mesmas técnicas em **APIs públicas reais**, na Internet.

Começaremos pela PokeAPI, uma API **aberta**, que não exige cadastro. Em seguida, passaremos para a REST Countries, que exige uma **chave de API** enviada no cabeçalho `Authorization`.

Objetivos da aula

Ao final da aula, você deverá ser capaz de:

- ler a documentação de uma API pública e montar as URLs das requisições;
- navegar em respostas JSON com listas e dicionários aninhados;
- usar parâmetros de consulta para paginar e filtrar resultados;
- autenticar requisições com uma chave de API guardada em variável de ambiente;
- tratar os erros mais comuns: 401, 403, 404, 429, `Timeout` e `ConnectionError`.

Etapa	API	O que vamos praticar
Parte 1	PokeAPI (aberta)	GET, JSON aninhado, paginação, erro 404
Parte 2	REST Countries (com chave)	Cabeçalho `Authorization`, variáveis de ambiente, `Session`, filtros e erros 401/429
Atividade	REST Countries	Script de consulta de países

> **Pré-requisito**
>
> Capítulo 1 da Unidade 2 (requests, JSON, parâmetros, cabeçalhos e tratamento de erros). Diferentemente da API local, aqui é preciso ter **acesso à Internet**.

0. Preparando o ambiente

Use o mesmo ambiente virtual da unidade ou crie um novo:

```
mkdir aula_apis
cd aula_apis
python -m venv .venv
source .venv/bin/activate        # Linux/macOS
# .venv\Scripts\Activate.ps1    # Windows (PowerShell)
python -m pip install requests
```

Parte 1 – PokeAPI: uma API aberta

A PokeAPI disponibiliza dados sobre Pokémon: espécies, tipos, habilidades, golpes, etc. Ela é somente leitura (apenas GET), não exige chave e responde sempre em JSON. A URL base é:

```
https://pokeapi.co/api/v2/
```

Os recursos seguem o padrão REST que estudamos: `/pokemon/{nome ou id}`, `/type/{nome}`, `/ability/{nome}` e assim por diante. Antes de programar, abra https://pokeapi.co/api/v2/pokemon/pikachu no navegador e observe o JSON retornado.

1.1 Primeira requisição

```
url = "https://pokeapi.co/api/v2/pokemon/pikachu"

resposta = requests.get(url, timeout=10)

print(resposta.status_code)
print(resposta.headers["Content-Type"])
print(len(resposta.text), "caracteres")
```

```
200
application/json; charset=utf-8
(o tamanho varia: a resposta tem dezenas de milhares de caracteres)
```

A resposta é grande porque traz *tudo* sobre o Pokémon. Na prática, nunca imprimimos o JSON inteiro: convertemos com `json()` e extraímos só o que interessa.

> **Por que timeout=10?**
>
> Na Unidade 2 usamos `timeout=5` porque a API estava na própria máquina. Na Internet a latência é maior e varia; um valor um pouco mais alto evita falsos erros, mas continua impedindo que o programa trave para sempre.

1.2 Navegando no JSON

Parte da estrutura retornada (resumida):

```
{
  "id": 25,
  "name": "pikachu",
  "height": 4,
  "weight": 60,
  "types": [
    { "slot": 1, "type": { "name": "electric", "url": "https://pokeapi.co/api/v2/type/13/" } }
  ],
  "abilities": [
    { "ability": { "name": "static", ... }, "is_hidden": false, "slot": 1 },
    { "ability": { "name": "lightning-rod", ... }, "is_hidden": true, "slot": 3 }
  ],
  "stats": [
    { "base_stat": 35, "stat": { "name": "hp", ... } },
    { "base_stat": 55, "stat": { "name": "attack", ... } },
    ...
  ],
  ...
}
```

Observe que `types`, `abilities` e `stats` são **listas de dicionários**, e que cada item guarda outro dicionário dentro. Para chegar ao nome do tipo, o caminho é `dados["types"][0]["type"]["name"]`.

```
import requests

resposta = requests.get("https://pokeapi.co/api/v2/pokemon/pikachu", timeout=10)
resposta.raise_for_status()
dados = resposta.json()

# height vem em decímetros e weight em hectogramas
altura_m = dados["height"] / 10
peso_kg = dados["weight"] / 10

tipos = [t["type"]["name"] for t in dados["types"]]
habilidades = [h["ability"]["name"] for h in dados["abilities"]]

print(f"#{dados['id']} {dados['name'].title()}")
print(f"Altura: {altura_m} m | Peso: {peso_kg} kg")
print("Tipos:", ", ".join(tipos))
print("Habilidades:", ", ".join(habilidades))
print("Atributos base:")
for s in dados["stats"]:
    print(f"  {s['stat']['name']:<16} {s['base_stat']:>3}")
```

```
Habilidades: static, lightning-rod
Atributos base:
  hp                35
  attack            55
  defense           40
  special-attack    50
  special-defense   50
  speed             90
```

> **Leia a documentação!**
>
> Foi a documentação que nos disse que `height` está em decímetros e `weight` em hectogramas. O JSON só traz os números; o significado deles está na documentação.

1.3 Parâmetros de consulta: paginação

Quando pedimos a coleção `/pokemon` sem identificador, a API devolve uma lista **paginada**. Os parâmetros `limit` (quantos itens) e `offset` (quantos pular) controlam a página, e podemos enviá-los com o argumento `params`, como fizemos na seção 1.4 da unidade:

```
import requests

url = "https://pokeapi.co/api/v2/pokemon"
parametros = {"limit": 5, "offset": 0}

resposta = requests.get(url, params=parametros, timeout=10)
resposta.raise_for_status()
pagina = resposta.json()

print(resposta.url)
print("Total de Pokémon:", pagina["count"])
for item in pagina["results"]:
    print("-", item["name"])
print("Próxima página:", pagina["next"])
```

```
https://pokeapi.co/api/v2/pokemon?limit=5&offset=0
Total de Pokémon: 1351
- bulbasaur
- ivysaur
- venusaur
- charmander
- charmeleon
Próxima página: https://pokeapi.co/api/v2/pokemon?offset=5&limit=5
```

O campo `next` já traz a URL da próxima página (ou `None` na última). O total pode mudar quando a API é atualizada.

1.4 Tratando erros

Se o recurso não existir, a PokeAPI responde **404 Not Found**. Vamos reunir o que aprendemos em uma função reutilizável:

```
BASE = "https://pokeapi.co/api/v2"


def buscar_pokemon(nome):
    url = f"{BASE}/pokemon/{nome.strip().lower()}"
    try:
        resposta = requests.get(url, timeout=10)
        resposta.raise_for_status()
        return resposta.json()
    except requests.exceptions.HTTPError:
        if resposta.status_code == 404:
            print(f"Pokémon '{nome}' não encontrado.")
        else:
            print("Erro HTTP:", resposta.status_code)
    except requests.exceptions.Timeout:
        print("A API demorou demais para responder.")
    except requests.exceptions.ConnectionError:
        print("Falha de conexão. Verifique sua Internet.")
    return None


for nome in ["Charizard", "pikachuu"]:
    dados = buscar_pokemon(nome)
    if dados:
        print(dados["name"], "->", dados["id"])
```

```
charizard -> 6
Pokémon 'pikachuu' não encontrado.
```

> **Uso consciente de APIs públicas**
>
> A PokeAPI é gratuita e mantida por voluntários. A documentação pede que os clientes guardem localmente (em *cache*) os dados já consultados, em vez de repetir as mesmas requisições. Evite laços que façam centenas de requisições sem necessidade.

Exercícios – Parte 1

- **Comparador:** peça dois nomes ao usuário e mostre qual dos dois tem o maior valor de `speed`.
- **Por tipo:** consulte `/type/electric` e liste os 10 primeiros Pokémon do tipo. Dica: os nomes estão em `dados["pokemon"][i]["pokemon"]["name"]`.
- **Paginação:** percorra as páginas seguindo o campo `next` até reunir os 60 primeiros nomes.

APIs abertas x APIs com chave

A PokeAPI não sabe quem está fazendo as requisições. Muitas APIs, porém, precisam identificar o cliente para controlar limites de uso, cobrar por planos ou bloquear abusos. Para isso, fornecem uma **chave de API** (*API key* ou *token*), enviada a cada requisição.

	PokeAPI	**REST Countries**
Cadastro	Não	Sim (plano gratuito)
Identificação do cliente	Nenhuma	Cabeçalho `Authorization: Bearer <chave>`
Sem credencial	Funciona normalmente	**401 Unauthorized**
Limites	Uso consciente (cache)	Limite mensal do plano e até 20 requisições a cada 10 s

Parte 2 – REST Countries: uma API que exige chave

A REST Countries fornece dados normalizados de mais de 250 países: nomes, códigos ISO, capitais, moedas, idiomas, fronteiras, bandeiras, população, etc. A versão atual (v5) tem a URL base:

```
https://api.restcountries.com/countries/v5
```

> **Atenção: versões antigas**